arXiv:2409.14674v1 [cs.RO] 23 Sep 2024

RACER: Rich Language-Guided Failure Recovery Policies for Imitation Learning

Yinpei Dai*[1], Jayjun Lee*[2], Nima Fazeli[2], Joyce Chai[1]

***Abstract*— Developing robust and correctable visuomotor policies for robotic manipulation is challenging due to the lack of self-recovery mechanisms from failures and the limitations of simple language instructions in guiding robot actions. To address these issues, we propose a scalable data generation pipeline that automatically augments expert demonstrations with failure recovery trajectories and fine-grained language annotations for training. We then introduce Rich languAge-guided failure reCovERy (RACER), a supervisor-actor framework, which combines failure recovery data with rich language descriptions to enhance robot control. RACER features a vision-language model (VLM) that acts as an online supervisor, providing detailed language guidance for error correction and task execution, and a language-conditioned visuomotor policy as an actor to predict the next actions. Our experimental results show that RACER outperforms the state-of-the-art Robotic View Transformer (RVT) on RLbench across various evaluation settings, including standard long-horizon tasks, dynamic goal-change tasks and zero-shot unseen tasks, achieving superior performance in both simulated and real world environments. Videos and code are available at: https://rich-language-failure-recovery.github.io.**

I. INTRODUCTION

Building reliable multi-task visuomotor policies for object manipulation through imitation learning is a long-standing challenge in robot learning. Recent advances in transformer-based architectures for imitation learning have demonstrated its effectiveness in 6-DoF Cartesian End-Effector (EE) control [1], [2], [3]. Despite this progress, current policies still suffer from an inability to self-recover from online failures during inference time [4], [5], [6]. Such limitation primarily stems from: (1) these models being trained exclusively on successful expert trajectories, without accounting for failures caused by model mispredictions and inevitable compounding errors, and (2) the absence of mechanisms to efficiently intervene and correct mistakes without requiring humans to take over the control via shared autonomy [7], [8].

To address these issues, prior works have focused on using human-in-the-loop interactive imitation learning to closely monitor and rectify robot behaviors through online language corrections [9], [10], [11]. However, these approaches impose a significant burden on human operators to manually intervene robot actions at inference time, and require collecting new online data for iterative model improvement. Moreover, these approaches typically rely on *simple* language instructions to guide robotic manipulation, such as *"move to the left"* and *"pick up the cup"*, which are insufficient for enabling robots to understand failures and take more accurate corrective actions. We argue that visuomotor policies should learn from failure recovery data paired with *rich* language instructions for effective online recovery. As shown in Fig. 1, to guide the robot to a proper recovery pose, language guidance should not only specify basic actions (e.g., predicates and objects) but also contain more sufficient information about failure analysis, spatial movement descriptions and the expected outcome of taking the current action. This richer, more descriptive language support is crucial for enhancing the robot's ability to comprehend complex scenarios, recover from errors, and ultimately improve the overall performance.

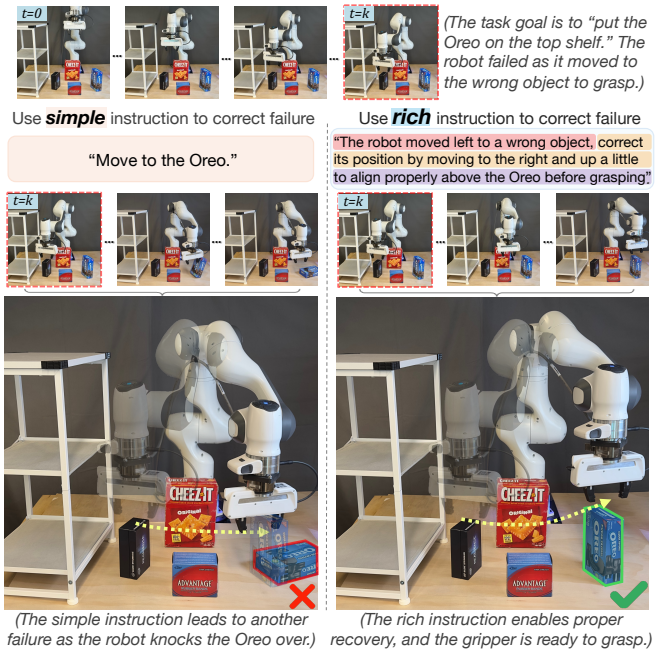

Fig. 1: Comparison between the *simple* and *rich* language guidance for failure recovery: The robot should approach the Oreo (the blue box on the right) directly to grasp it but instead moved to the wrong object (the black box). To help the visuomotor policy recover from this failure, the rich language instruction provides sufficient details, including a failure analysis (in red), spatial movements (in orange) and the expected outcome (in purple). In contrast, simple language instructions with limited descriptions may not guide the robot effectively, potentially causing it to continue making mistakes.

However, most existing popular benchmarks lack either failure recovery data or trajectories paired with rich language descriptions [12], [13], [14], [15]. Therefore, we propose an automatic failure data augmentation pipeline that extends expert demonstrations from RLBench [12] by using ran-

[1]Computer Science and Engineering Deparment, [2]Robotics Department, University of Michigan, MI, USA. * denotes equal contribution.
Emails: {daiyp,jayjun,nfz,chaijy}@umich.edu
This work is supported in part by NSF IIS1949634, NSF SES-2128623, and has benefited from the Microsoft Accelerate Foundation Models Research (AFMR) grant program.

dom perturbations to generate failure data and leveraging large language models (LLMs) to annotate rich language descriptions for each transition. We then introduce RACER, a flexible *supervisor-actor* framework that enhances robotic manipulation through rich language guidance for failure recovery. RACER consists of a vision-language model (VLM) as the *supervisor*, which monitors and provides detailed instructions to analyze, correct, and guide robot actions at each step, and a language-conditioned visuomotor policy as the *actor*, responsible for generating the next appropriate action. Our experiments show that RACER significantly outperforms previous state-of-the-art baselines across 18 RLBench tasks, demonstrating superior robustness and adaptability to task goal online changes, unseen task evaluations, and real world scenarios. We summarize our contributions as follows:

- We are the first to explore the role of rich language guidance in robot manipulation and demonstrate its importance in conjunction with failure recovery for robust control.
- We propose a scalable language-guided failure recovery data augmentation strategy and collect 10,159 new trajectories with rich language instructions on RLbench.
- We present RACER, where a VLM instructs a visuomotor policy with rich language. RACER performs competitively on RLbench, demonstrating robustness to task goal changes and generalizability in handling unseen tasks.
- We show that RACER enables fast real-world deployment through few-shot sim-to-real transfer, highlighting the role of rich language guidance in bridging the sim-to-real gap.

II. Related Works

Imitation Learning for Visuomotor Policies. Imitation learning is commonly used to train visuomotor policies with the supervision of expert demonstrations [16], [17], predicting actions on either sparse keyframes [1], [18] or dense waypoints [19], [20]. However, this approach often struggles with out-of-distribution observations, such as failure states. To address this, many methods utilize human-in-the-loop interactive learning, where humans need to monitor and intervene using shared autonomy [7], [8] or language corrections [9], [11], [10]. In contrast, we propose an automated failure recovery pipeline that augments existing expert demonstrations into rich language-annotated trajectories, enhancing 3D robotic manipulation and few-shot sim-to-real transfer.

Failure Detection and Recovery. Self-recovery from online failures is crucial for robots. Prior works leverage external failure detectors—either trained models [8], [21] or proprietary LLMs [22], [23]—to monitor performance and request human assistance when needed. Other approaches focus on collecting failure recovery trajectories through scalable auto-generation pipelines [6], [24], [25]. For example, I-Gen [6] builds on MimicGen [15] to automatically generate corrective interventions from a small set of human demos to cover more failed states. Similarly, [25] uses LLMs to verify the robot's internal information and retry tasks until successful. However, these methods do not incorporate language-guided control, limiting their ability to adapt or shape robot's behavior based on human instructions or linguistic feedback.

Language-guided Robot Learning. Language is a natural medium for humans to specify tasks and to interact with robots. Most previous works [26], [1], [3] have focused on using short task goal descriptions to instruct multi-task visuomotor policies. Recent works have been shifting towards enabling real-time human intervention through verbal corrections. For example, OLAF [10] uses GPT-4 to re-label incorrect actions based on user feedback like *"move closer to the cup"*. RT-H [27] employs the RT-2 model [28] to generate language instructions and robot action tokens within predefined hierarchies, allowing for human interventions in a fixed set of spatial movements. Similarly, YAY Robot [11] trains a high-level language policy to adjust behaviors on-the-fly by retrieving instructions from a candidate pool and a low-level policy to follow these instructions. However, unlike YAY and other approaches that rely on simple instructions (usually a verb and a noun), our work leverages rich language descriptions, including failure analysis, fine-grained spatial movements, and expected outcomes. Furthermore, our models are trained directly on augmented failure recovery data, reducing the need for additional online data collection to cover failure states and recovery actions.

III. Method

We develop a data augmentation pipeline to produce rich language-guided failure recovery trajectories and a framework named RACER, where a VLM (*supervisor*) guides a visuomotor policy (*actor*) for robust robotic manipulation.

A. Problem Statement

Our task is language-conditioned robotic control with two sub-problems: (1) language-conditioned multi-task imitation learning for visuomotor policies and (2) single-view image-conditioned language instruction generation for VLMs. Assume we have a multi-task dataset of expert demonstrations $\mathcal{D}^{\text{expert}}$ containing successful trajectories, where each trajectory $\tau = (\delta_1, \delta_2, \cdots, \delta_T, L)$ consists of a sequence of waypoint transitions $\delta_{1:T}$ obtained via heuristic keyframe discovery [26] and a high-level task goal L expressed in natural language like *"close the red jar"*. Each transition δ_t is a tuple of (o_t, a_t, o_{t+1}) at timestep t, where o_t is the observation from RGB-D cameras and proprioceptive states and a_t is a 9-dim waypoint action including a 6-DoF EE pose, a binary gripper state, and a binary indicator for planning a collision-free path. A common objective for visuomotor policies is to learn $\pi(a_t|o_t, L)$ from $\mathcal{D}^{\text{expert}}$. However, training policies solely on task goal L often leads to overfitting to demonstrations, resulting in poor language understanding and instruction-following [11], [25], [29]. Therefore, we introduce rich language instruction ℓ_t for each keyframe transition to train a better visuomotor policy $\pi(a_t|o_t, \ell_t, L)$. Note that unlike the previous works [11], [27] that only use simple instructions and require human intervention, we aim to generate more expressive sentences for better language control in an **automated** way without the necessity of human involvement for failure correction. In addition, we fine-tune a VLM $p_{\text{vlm}}(\ell_t|o_t, \ell_{t-1}, L)$ for explaining failure states

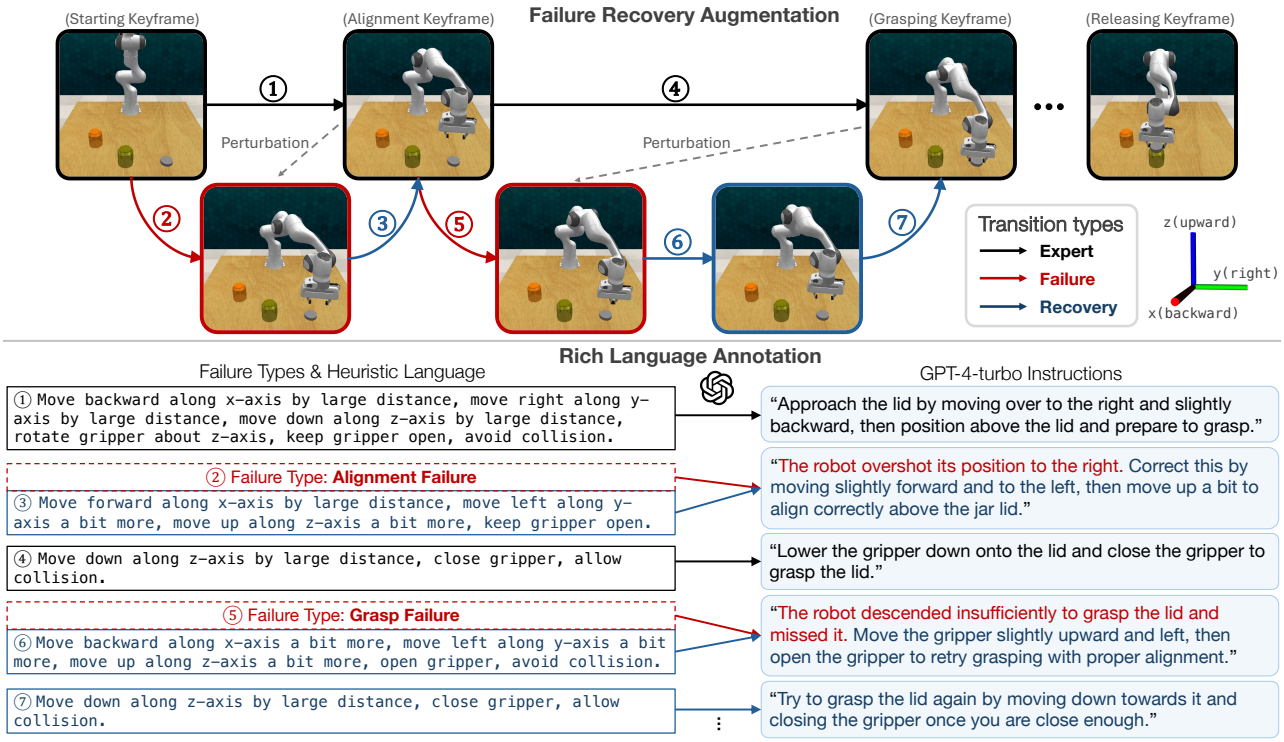

Fig. 2: An overview of automatic rich language-annotated failure-recovery data augmentation pipeline. Given an expert demo (e.g., task goal: *close the olive jar*), perturbations are injected to expert actions at crucial keyframes (e.g. aligning to, grasping, and releasing a target object) to induce failures. Then, the expert actions are reused as corrections to collect recovery transitions. Finally, all expert and recovery transitions are labelled with rich instructions through GPT-4-turbo. The input for GPT-4-turbo includes the task description, ground-truth object locations, failure types, and heuristic language describing the change in the end-effector's pose movement at the current step.

and generating rich language instructions given the current observations, previous instructions and high-level task goals.

B. Data Generation

1) Failure Definition: We define failure as a significant deviation from the expert action at a given keyframe, classified into two categories: (1) *recoverable failure*, correctable by using existing expert actions, and (2) *catastrophic failure*, requiring a scene reset due to excessive scene state changes (e.g., objects being knocked over or falling off the table).

2) Failure Recovery Augmentation: we aim to scalably extend existing expert demonstrations with recoverable failures without additional human efforts. For each trajectory τ in $\mathcal{D}^{\text{expert}}$, we identify and perturb a set of *crucial keyframes* that correspond to motion primitives for alignment (e.g., move to the above of objects), grasping (e.g., lower down to pick), and releasing (e.g. place the grasped object down). Heuristic rules based on the gripper's opening state, positional changes, and timestep number are used to determine the crucial keyframes. Fig. 2 illustrates the data augmentation strategy in detail. To be concrete, suppose there is a crucial keyframe at timestep j with an expert transition $\delta_{j-1} = (o_{j-1}, a_{j-1}, o_j)$ from the previous timestep. We then add truncated Gaussian noise to randomly perturb the expert action a_{j-1} into $\tilde{a}_{j-1} = a_{j-1} + \epsilon$ and step through the environment to get the failure transition $\tilde{\delta}_{j-1} = (o_{j-1}, \tilde{a}_{j-1}, \tilde{o}_j)$ to cover failure states (see step ② and ⑤ in Fig.2). The expert action a_{j-1} can be used as the correction to get recovery transition $\delta^{\text{c}}_j = (\tilde{o}_j, a^{\text{c}}_j, o^{\text{c}}_{j+1})$, where $a^{\text{c}}_j = a_{j-1}$ and $o^{\text{c}}_{j+1} = o_j$ (see step ③ in Fig. 2). During our experiment, we found that such *one-step* recovery strategy works for alignment failures, but is not adequate for more complex motion primitives, such as grasping and releasing, as an immediate recovery will lead to catastrophic collision with target objects. Therefore, we also propose a *two-step* recovery strategy (see step ⑥ and ⑦ in Fig.2) with an intermediate transition $\delta^{\text{i}}_j = (\tilde{o}_j, a^{\text{i}}_j, o^{\text{i}}_{j+1})$ added before the recovery transition $\delta^{\text{c}}_{j+1} = (o^{\text{i}}_{j+1}, a^{\text{c}}_{j+1}, o^{\text{c}}_{j+2})$, where $a^{\text{c}}_{j+1} = a_{j-1}, o^{\text{c}}_{j+2} = o_j$, and a^{i}_j is an expert action sampled from the waypoints in between keyframes $j-1$ and j. After the augmentation, all episodes containing failure recovery are rolled out and filtered based on task success. Finally, we obtain a new dataset $\mathcal{D}^{\text{recovery}}$ that includes additional recovery transitions δ^{i} and δ^{c}. This procedure significantly offloads human efforts for monitoring the policy online and intervening to correct failures as in [11], [27].

3) Rich Language Annotation: It is quite challenging for LLMs to generate faithful language descriptions based on numerical action values due to the notorious hallucination problem [30]. Therefore, we include detailed task descriptions, ground-truth object locations, failure types and heuristic language to construct informative prompts for LLMs. Specifically, the heuristic language consists of template-based natural descriptions of the EE pose movement caused by the last action (see the left bottom part in Fig. 2), in which we compare the state changes between the last and current observations in terms of position, orientation, gripper state, and collision. Based on this information, we query

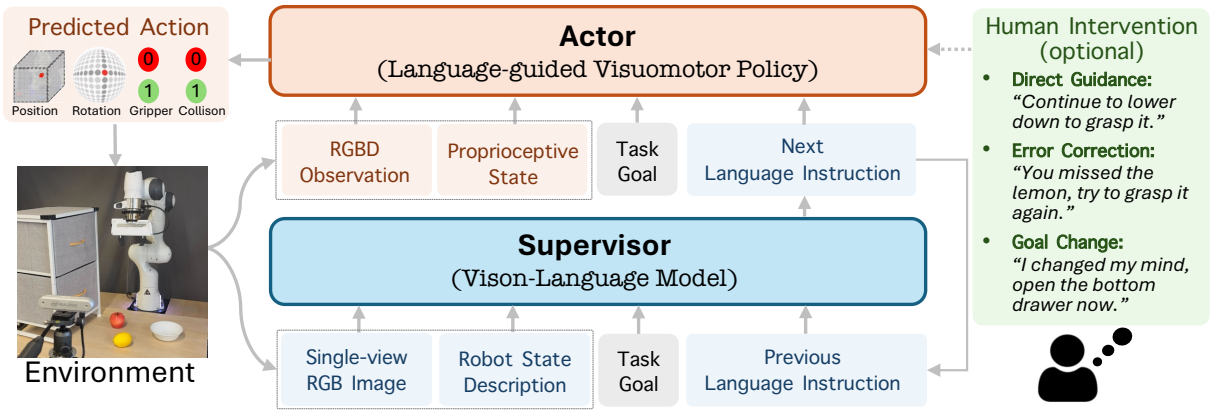

Fig. 3: The RACER framework consists of: (1) the *Supervisor*, a VLM that monitors the robot's behavior, providing feedback for task execution and error correction with rich instructions; and (2) the *Actor*, a language-conditioned visuomotor policy that generates actions based on visual observations, proprioceptive states, and language guidance that includes a high-level task goal and an instruction.

GPT-4-turbo to paraphrase the heuristic language into more coherent and expressive natural language. After $\mathcal{D}^{recovery}$ is annotated with rich instructions, we obtain our language-guided failure-recovery dataset $\mathcal{D}^{recovery+lang}$, where each transition is represented as $\delta_t = (o_t, a_t, \ell_t, o_{t+1})$ and ℓ_t refers to the rich instruction that guides the visuomotor policy.

C. Model

RACER consists of two main parts (Fig. 3): (1) a VLM as the supervisor to generate rich language instruction ℓ_t; and (2) a language-conditioned visuomotor policy as the actor to predict the action a_t to interact with the environment.

1) Supervisor: In our experiment, we select the front-view RGB image as the visual input. The robot state description summarizes the change of the robot proprioceptive state after taking the last action. We compare the delta changes in position, rotation, gripper state and collision to generate the description based on predefined templates, e.g., "*The robot moved (forward|backward|downward|upward|left|right) (a little|significantly)*". Then, we prepare the input for VLM in a format of "*<image>\nThe task goal is: {task_goal}. In the previous step, the robot arm was given the following instruction: {previous_instruction}. {robot_state_description}. Based on the visual observation and the context, how does the robot fulfill that previous instruction and what's the next instruction for the robot?*" to generate the next instruction.

2) Actor: The actor is a language-guided visuomotor policy, where any existing waypoint-based models such as RVT [3] and PerAct [1] can be used interchangeably. To enhance the capability of these models to understand rich language instructions, we concatenate the high-level task goal L and the rich instruction ℓ_t together as the language input for the policy at each step in the following format as *"Task goal: {task_goal}.\nCurrent instruction: {rich_instruction}"*, in which the {rich_instruction} is generated by the VLM. After adapting to our augmented data, visuomotor polices can predict actions based on more descriptive language.

3) Online Evaluation: During evaluation, for each step, we first use the VLM supervisor to analyze current scene (i.e, determine whether the robot started the task or followed the last instruction correctly or made a recoverable failure) and generate a fine-grained instruction, which is then used to instruct the actor to predict an action to control the robot.

IV. Experiments

A. Experimental set-up

We choose RLbench [12] as our benchmark for simulated experiments and test sim-to-real transfer on a Panda robot.

1) Model backbone: For the supervisor, we select llama3-llava-next-8B [31], a latest variant of LLaVA model [32] due to its superior multimodal reasoning capabilities and the simplicity of fine-tuning under a limited budget. For the actor, we choose RVT [3], one of the state-of-the-art visuomotor policy, and adtrain on our new dataset.

2) Augmented Dataset: We gather the training and validation expert demos from RLbench as $\mathcal{D}^{\text{expert}}$ (2250 episodes in total), perturb each episode five times and filter unsuccessful trajectories to obtain $\mathcal{D}^{\text{recovery+lang}}$ (10,159 episodes in total). Both simple and rich language instructions are generated by prompting GPT-4-turbo for comparative study. The simple instructions resemble previous works [27], [11], consisting of a short sentence that mainly includes a verb (e.g., *move*) and a noun (e.g., *jar*) or direction (e.g., *left*), whereas the rich instructions include more failure explanation, detailed descriptions for the spatial movements, attributes (e.g. color, location and shape) about target objects, and the expected outcome of taking the action. Table II compares the richness of language across different datasets, where we measure the average sentence length, the number of semantic roles [33] and unique semantic tags using the AllenNLP toolkit [34].

3) Training Details: For the supervisor, we use LoRA [35] to continually fine-tune the LLaVA model for 2 epochs, with a LoRA rank of 128 and a scaling factor α of 256. To stabilize training, we use deepspeed zero2 stage [36] with a batch size of 64 and a learning rate of 2e-5. For the actor, we modify RVT by replacing its original language encoder CLIP [37] with T5-11B [38] to enhance its language understanding and removing the timestep input from the proprioceptive state as we find it hinders the language controllability of the policy. We use the LAMB optimizer [39] to train the modified RVT is for 18 epochs, with a batch size of 48 and

Models	Avg. Succ. (↑)	Avg. Rank (↓)	Close Jar	Drag Stck	Insert Peg	Meat off Grill	Open Drawer	Place Cups	Place Wine	Push Buttons
PerAct [1]	49.4	3.7	55.2 ± 4.7	89.6 ± 4.1	5.6 ± 4.1	70.4 ± 2.0	88.0 ± 5.7	2.4 ± 3.2	44.8 ± 7.8	92.8 ± 3.0
RVT [3]	62.9	2.2	52.0 ± 2.5	**99.2** ± 1.6	11.2 ± 3.0	88.0 ± 2.5	71.2 ± 6.9	4.0 ± 2.5	91.0 ± 5.2	**100.0** ± 0.0
Act3D [2]	65.0	2.2	**92.0**	92.0	**27.0**	**94.0**	93.0	3.0	80.0	99.0
RACER (Ours)	**70.2** ± 1.13	**1.6**	85.6 ± 2.0	**99.2** ± 1.6	9.6 ± 4.8	91.2 ± 3.0	**100.0** ± 0.0	**6.4** ± 4.1	**98.4** ± 2.0	**100.0** ± 0.0
RACER+H (Ours)	80.1 ± 0.52	–	91.2 ± 1.6	100.0 ± 0.0	25.6 ± 5.4	98.4 ± 2.0	100.0 ± 0.0	6.4 ± 4.1	100.0 ± 0.0	100.0 ± 0.0
Models	**Put in Cupboard**	**Put in Drawer**	**Put in Safe**	**Screw Bulb**	**Slide Block**	**Sort Shape**	**Stack Blocks**	**Stack Cups**	**Sweep to Destpan**	**Turn Tap**
PerAct [1]	28.0 ± 4.4	51.2 ± 4.7	84.0 ± 3.6	17.6 ± 2.0	74.0 ± 13.0	16.8 ± 4.7	26.4 ± 3.2	2.4 ± 2.0	52.0 ± 0.0	88.0 ± 4.4
RVT [3]	49.6 ± 3.2	88.0 ± 5.7	91.2 ± 3.0	48.0 ± 5.7	81.6 ± 5.4	**36.0** ± 2.5	**28.8** ± 3.9	26.4 ± 8.2	72.0 ± 0.0	93.6 ± 4.1
Act3D [2]	**51.0**	90.0	**95.0**	47.0	93.0	8.0	12.0	9.0	**92.0**	**94.0**
RACER (Ours)	50.4 ± 4.1	**100.0** ± 0.0	93.6 ± 4.1	**72.0** ± 5.7	**99.2** ± 1.6	25.6 ± 4.1	15.2 ± 3.0	**41.6** ± 5.4	84.0 ± 0.0	91.2 ± 3.0
RACER+H (Ours)	71.2 ± 6.9	100.0 ± 0.0	100.0 ± 0.0	92.0 ± 0.0	100.0 ± 0.0	38.4 ± 2.0	60.0 ± 5.7	69.6 ± 4.1	89.6 ± 2.0	100.0 ± 0.0

TABLE I: Multi-task performance comparision of different models on 18 RLbench tasks. RACER+H is RACER with human intervention.

Dataset	Length	# Semantic Roles	# Unique Tags	Example
RT-H [27]	4.52	1.06	2.26	*"move arm left"*
YaY [11]	4.73	1.04	3.79	*"move to the left"*
Ours (simple)	4.38	1.06	2.69	*"move left"*
Ours (rich)	18.28	3.64	8.31	*"It moved too right, correct it position by moving slightly left, then align with ⋯"*

TABLE II: Comparison of language richness levels for datasets.

a learning rate of 1.5e-3. All training processes take around 30 hours to finish with 8 40GB A40 GPUs.

B. Simulated Experiments

Following the multi-task learning setup in [1], we evaluate our framework over 18 RLbench tasks with a total of 450 episodes. Visual observations are captured from four RGB-D cameras positioned at the front, left shoulder, right shoulder, and wrist. The front-view images are input to the VLM, while all views are input to the visuomotor policy.

1) Baselines: We compare with the following baselines: (1) PerAct [1] encodes the RGB-D images into a sequence of voxel grid patches and uses the perceiver transformer [40] to predict actions; (2) RVT [3] projects the pointcloud into multiple virtual images from orthogonal perspectives and aggregates information across the views via a transformer. As RVT takes images rather than voxels as input, it scales and performs better than PerAct while achieving faster training and inference speed; (3) Act3D [2] lifts pre-trained 2D CLIP features into 3D using depth sensing, and learns a 3D scene feature field through recurrent coarse-to-fine point sampling and relative-position attention to decode optimal EE actions.

2) Multi-task Performance: Table I summarizes the performance comparisons between RACER trained with rich instructions and the aforementioned baselines. All results are averaged over 5 random seeds on 18 RLbench tasks. When guided by VLM-generated rich instructions, RACER achieves a significant improvement, with an average success rate of 70.2%. This marks a 7.3% increase over RVT (62.9%) and a 5.2% gain over Act3D (65.0%). For some long-horizon tasks, such as Put in Drawer, Screw Bulb and Stack Cups, RACER outperforms baselines by 10-24%. To further comprehensively understand RACER performance, we design experiments to study several questions as follows.

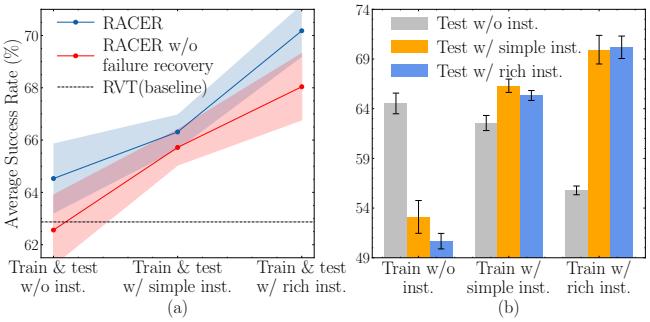

Fig. 4: (a) Comparison of RACER's performance trained with and without failure recovery across three types of instructions. (b) Cross-evaluation of RACER trained with failure recovery, where training and testing were conducted on varying instruction types.

How does the language richness and failure recovery behaviors improve overall performance? To answer this question, we train RACERs with three types of language input: (1) *no inst.*, i.e., only task goals are provided, no additional instructions; (2) *simple inst.*, i.e., both task goals and simple instructions are given; and (3) *rich inst.*, i.e., both task goals and rich instructions are given. Additionally, we ablate these models by training without any failure recovery transitions. For each model, the training and testing conditions are set to be the same. As shown in Fig. 4(a), both failure recovery transitions and rich language inputs substantially improve performance, highlighting their importance for robot manipulation. With richer language input, performance consistently increases, and failure recovery data generally boosts performance (around 2%) across all language settings.

Can RACER trained with rich instructions still perform well given only simple instructions? We conduct a cross-evaluation of RACERs by training and testing them on varied language settings. From Fig. 4(b), we surprisingly find that the policy trained with rich instructions is still quite robust to simple instructions during testing, substantially outperforming policies trained and tested both with simple instructions (66.31%→69.94%), which underscores the importance and generalizability of rich language training paradigm. When evaluating without any language instructions, the policy degrades drastically due to the severe mismatch between training and testing conditions. However, it should be noted that our RACER uses a VLM to automatically generate desired instructions without human efforts, thus can circumvent the low performance issue brought by no instruction setting.

Method	Avg. Succ. (Goal Change)	Close Jar	Screw Bulb	Open Drawer	Push Buttons
RVT [3]	9.0	0.0	20.0	16.0	0.0
RACER	**60.0**	**64.0**	**40.0**	**80.0**	**56.0**
Method	Avg. Succ. (Unseen Task)	Close Drawer	Move Block	Reach Target	Pick up Cup
RVT [3]	16.0	16.0	4.0	20.0	24.0
RACER	**47.0**	**68.0**	**32.0**	**40.0**	**48.0**

TABLE III: Results for goal change tasks and unseen tasks.

What is the upper bound performance of RACER when humans can intervene? We further conduct human intervention experiments (see RACER+H in Tab. I) where humans can decide to modify VLM instructions as needed (e.g., when the VLM generates hallucinated or inappropriate instructions) with their own, often *simple*, instructions. This approach increases the success rate from 70.2% to 80.1% with only a 24% intervention rate among the total steps, which demonstrates that our policy, trained on rich instructions, effectively understands and follows unseen human commands as well.

3) Task Goal Online Change Experiments: We introduce a novel setting to assess the model's robustness by deliberately switching the task goal during execution. For example, in the case of Fig. 2, we may instruct the robot to place the grasped lid on a different jar just before it is about to release the lid onto the correct jar. The original high-level task goal is replaced with a new one (e.g., *"close the orange jar"*), and a short sentence describing the updated goal is provided to the robot (e.g., *"No, I changed my mind, move to the orange jar instead."*). After this intervention, no further human instructions are allowed and models need to finish the new task goal on its own. We select four tasks from RLbench for testing, each with 25 variations. As shown in the upper part of Table III, RACER achieves 60 successful episodes out of 100 trials, significantly outperforming RVT, indicating its robustness in handling dynamic goal change scenarios.

4) Unseen Task Experiments: We evaluate our models on unseen tasks to examine RACER's zero-shot adaptability. Four new tasks are selected from the RLbench suite, where the objects and manipulation skills may be similar to the training data, but the combinations and scenes are novel. Each task is tested with 25 variations, with results shown in the lower part of Table III. Compared to RVT, RACER performs significantly better, demonstrating its generalizability to generate appropriate instructions based on images from unseen scenarios and execute reasonable actions accordingly.

C. Real World Experiments

We evaluate our model in a real-world setup using a 7-DoF Franka Emika Panda robot and a statically mounted front-view Realsense D455 RGB-D camera. Pointclouds are obtained from the camera as model inputs after extrinsic calibration. We choose four tasks (Open Drawer, Place Fruits, Push Buttons, Put Item on Shelf) for testing and collect 60 training demos (15 per task) with failure augmentation (three perturbations each) via manual kinesthetic guidance and GPT-4-turbo language annotation. Both the visuomotor policy in RACER and the RVT are fine-tuned for 15 epochs with a learning rate of 1.5e-3 and a batch size of 24, while LLaVA is fine-tuned for 2 epochs with a learning rate of 1e-5 and a batch size of 32. We compare four ablated models: (1) $\text{RACER}_{scratch}$, trained from scratch on real data; (2) $\text{RACER}_{no\ inst.}$, trained without instructions on simulated and real data; (3) $\text{RACER}_{simple\ inst.}$, trained with simple instructions on simulated and real data; and (4) $\text{RACER}_{w/o\ FA}$, trained with rich instructions but without failure analysis. All models are tested on 40 episodes (10 per task, where 7 are for regular tasks and 3 involves task goal changes).

Models	Avg. Succ.	Open Drawer	Pick and Place Fruits	Push Buttons	Put Item on Shelf
RVT [3]	25.0	10.0	30.0	20.0	40.0
$\text{RACER}_{scratch}$	32.5	50.0	10.0	30.0	40.0
$\text{RACER}_{no\ inst.}$	25.0	60.0	0.0	0.0	40.0
$\text{RACER}_{simple\ inst.}$	32.5	60.0	10.0	20.0	40.0
$\text{RACER}_{w/o\ FA}$	62.5	**70.0**	40.0	80.0	60.0
RACER	**72.5**	**70.0**	**50.0**	**100.0**	**70.0**

TABLE IV: Results for real world tasks.

As shown in Table IV, RACER demonstrates a significant improvement over RVT, achieving a 47.5% higher overall success rate. This highlights the crucial role of integrating rich descriptions and failure recovery data. Notably, RACER also substantially outperforms $\text{RACER}_{scratch}$, showing the effectiveness of pre-training in simulation with rich language, which leads to superior sim-to-real transfer. When comparing RACER with variants trained with different instruction settings, we observe a steady improvement as the language richness increases, suggesting that more expressive language helps bridge the sim-to-real gap [41] for few-shot adaptation. Additionally, during experiments, we found that RVT and RACERs trained with simple or no instructions exhibited weaker task understanding, often displaying repetitive behaviors across different tasks (e.g. grasping a drawer handle during the Push Buttons task as timesteps increased), indicating substantial overfitting to the training scenes. In contrast, training with rich language acts as a form of regularization, preventing overfitting and enabling RACER to achieve more robust control, better failure recovery, and improved adaptation to task-goal changes and scene variations.

V. Conclusion and Discussions

In this work, we present a scalable language-guided failure recovery data augmentation strategy and introduce RACER, a self-recoverable behavior adaptation framework driven by rich language guidance for robotic manipulation. Through joint training with rich language instructions and failure recovery data, RACER demonstrates strong performance and robustness across both simulated and real-world environments. However, our method currently relies on expert demos for data curation and keyframe extraction for sparse waypoint prediction. In the future, we plan to enhance our data pipeline by augmenting trajectories from human videos and incorporate dense waypoint policies for more precise control. Additionally, we aim to improve RACER's grounding abilities for better instruction generation and enable it to proactively ask clarifying questions when faced with ambiguity. These enhancements will further strengthen RACER's effectiveness and performance in handling complex real-world scenarios.

REFERENCES

[1] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Proceedings of the 6th Conference on Robot Learning (CoRL)*, 2022.

[2] Theophile Gervet, Zhou Xian, Nikolaos Gkanatsios, and Katerina Fragkiadaki. Act3d: 3d feature field transformers for multi-task robotic manipulation. In *7th Annual Conference on Robot Learning*, 2023.

[3] Ankit Goyal, Jie Xu, Yijie Guo, Valts Blukis, Yu-Wei Chao, and Dieter Fox. Rvt: Robotic view transformer for 3d object manipulation. 2023.

[4] Zeyi Liu, Arpit Bahety, and Shuran Song. Reflect: Summarizing robot experiences for failure explanation and correction. *CoRL*, 2023.

[5] Huihan Liu, Soroush Nasiriany, Lance Zhang, Zhiyao Bao, and Yuke Zhu. Robot learning on the job: Human-in-the-loop autonomy and learning during deployment. *RSS*, 2019.

[6] Ryan Hoque, Ajay Mandlekar, Caelan Garrett, Ken Goldberg, and Dieter Fox. Intervengen: Interventional data generation for robust and data-efficient robot imitation learning. *arXiv preprint arXiv:2405.01472*, 2024.

[7] Siddharth Karamcheti, Megha Srivastava, Percy Liang, and Dorsa Sadigh. Lila: Language-informed latent actions. In *Conference on Robot Learning*, pages 1379–1390. PMLR, 2022.

[8] Huihan Liu, Shivin Dass, Roberto Martín-Martín, and Yuke Zhu. Model-based runtime monitoring with interactive imitation learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4154–4161. IEEE, 2024.

[9] Yuchen Cui, Siddharth Karamcheti, Raj Palleti, Nidhya Shivakumar, Percy Liang, and Dorsa Sadigh. No, to the right: Online language corrections for robotic manipulation via shared autonomy. In *Proceedings of the 2023 ACM/IEEE International Conference on Human-Robot Interaction*, pages 93–101, 2023.

[10] Huihan Liu, Alice Chen, Yuke Zhu, Adith Swaminathan, Andrey Kolobov, and Ching-An Cheng. Interactive robot learning from verbal correction, 2023.

[11] Lucy Xiaoyang Shi, Zheyuan Hu, Tony Z Zhao, Archit Sharma, Karl Pertsch, Jianlan Luo, Sergey Levine, and Chelsea Finn. Yell at your robot: Improving on-the-fly from language corrections. *CORL*, 2024.

[12] Stephen James, Zicong Ma, David Rovick Arrojo, and Andrew J Davison. Rlbench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 5(2):3019–3026, 2020.

[13] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.

[14] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on robot learning*, pages 1094–1100. PMLR, 2020.

[15] Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Iretiayo Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. *CoRL*, 2023.

[16] Ahmed Hussein, Mohamed Medhat Gaber, Eyad Elyan, and Chrisina Jayne. Imitation learning: A survey of learning methods. *ACM Comput. Surv.*, 50(2), apr 2017.

[17] Yuke Zhu, Ziyu Wang, Josh Merel, Andrei Rusu, Tom Erez, Serkan Cabi, Saran Tunyasuvunakool, János Kramár, Raia Hadsell, Nando de Freitas, et al. Reinforcement and imitation learning for diverse visuomotor skills. *arXiv preprint arXiv:1802.09564*, 2018.

[18] Stephen James and Andrew J. Davison. Q-attention: Enabling efficient learning for vision-based robotic manipulation, 2022.

[19] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[20] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alex Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *arXiv preprint arXiv:2307.15818*, 2023.

[21] Cristina Cornelio and Mohammed Diab. Recover: A neuro-symbolic framework for failure detection and recovery. *IROS*, 2024.

[22] Yinpei Dai, Run Peng, Sikai Li, and Joyce Chai. Think, act, and ask: Open-world interactive personalized robot navigation. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3296–3303. IEEE, 2024.

[23] Zhi Zheng, Qian Feng, Hang Li, Alois Knoll, and Jianxiang Feng. Evaluating uncertainty-based failure detection for closed-loop LLM planners. In *ICRA 2024 Workshop on Back to the Future: Robot Learning Going Probabilistic*, 2024.

[24] Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha Srinivasa, and Abhishek Gupta. Ccil: Continuity-based data augmentation for corrective imitation learning. *ICLR*, 2024.

[25] Huy Ha, Pete Florence, and Shuran Song. Scaling up and distilling down: Language-guided robot skill acquisition. In *Conference on Robot Learning*, pages 3766–3777. PMLR, 2023.

[26] Stephen James, Kentaro Wada, Tristan Laidlow, and Andrew J. Davison. Coarse-to-fine q-attention: Efficient learning for visual robotic manipulation via discretisation, 2022.

[27] Suneel Belkhale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quon Vuong, Jonathan Tompson, Yevgen Chebotar, Debidatta Dwibedi, and Dorsa Sadigh. Rt-h: Action hierarchies using language. *arXiv preprint arXiv:2403.01823*, 2024.

[28] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[29] Ran Xu, Yan Shen, Xiaoqi Li, Ruihai Wu, and Hao Dong. Naturalvlm: Leveraging fine-grained natural language for affordance-guided visual manipulation. *arXiv preprint arXiv:2403.08355*, 2024.

[30] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. Hallucination is inevitable: An innate limitation of large language models. *arXiv preprint arXiv:2401.11817*, 2024.

[31] Bo Li, Kaichen Zhang, Hao Zhang, Dong Guo, Renrui Zhang, Feng Li, Yuanhan Zhang, Ziwei Liu, and Chunyuan Li. Llava-next: Stronger llms supercharge multimodal capabilities in the wild, May 2024.

[32] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36, 2024.

[33] Daniel Gildea and Daniel Jurafsky. Automatic labeling of semantic roles. *Computational linguistics*, 28(3):245–288, 2002.

[34] Matt Gardner, Joel Grus, Mark Neumann, Oyvind Tafjord, Pradeep Dasigi, Nelson F. Liu, Matthew Peters, Michael Schmitz, and Luke Zettlemoyer. AllenNLP: A deep semantic natural language processing platform. In Eunjeong L. Park, Masato Hagiwara, Dmitrijs Milajevs, and Liling Tan, editors, *Proceedings of Workshop for NLP Open Source Software (NLP-OSS)*, pages 1–6, Melbourne, Australia, July 2018. Association for Computational Linguistics.

[35] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[36] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE, 2020.

[37] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.

[38] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

[39] Yang You, Jing Li, Sashank Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, Kurt Keutzer, and Cho-Jui Hsieh. Large batch optimization for deep learning: Training bert in 76 minutes. *ICLR*, 2020.

[40] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. Perceiver: General perception with iterative attention. In *International conference on machine learning*, pages 4651–4664. PMLR, 2021.

[41] Albert Yu, Adeline Foote, Raymond Mooney, and Roberto Martín-Martín. Natural language can help bridge the sim2real gap. *CoRL*, 2024.